

Team Name : RAGnarok

Team Leader Name : Krishna H Zalavadiya

Problem Statement : Standard keyword search and single-path dense retrieval fail to surface “Tier-5 candidates” — people who are genuinely qualified but express their skills differently. RAGnarok solves this with 5 independent retrieval paths + layered scoring + LLM reasoning, ranking 100,000 candidates under 5 minutes on CPU only.

Solution Overview

RAGnarok: Intelligent Candidate Ranking at Scale

- End-to-end pipeline ranking 100,000 candidates against a job description in under 5 minutes on CPU — no GPU, no internet during ranking
- Fuses FIVE independent retrieval paths (Semantic + BM25 + Ontology + Trajectory + Signal) via Reciprocal Rank Fusion to eliminate single-method bias
- Goes beyond keyword matching: rewards real experience, career trajectories, and domain-transfer skills through a layered scoring + reasoning pipeline
- Solves the Tier-5 Rescue Problem — surfaces candidates who built equivalent systems under different domain names (e.g. RecSys → IR, NLP → Ranking)

Key Differentiators vs. Traditional Systems:

- Traditional: single-path keyword/vector search → misses contextual equivalents and domain transfers
- RAGnarok: 5-path hybrid retrieval → rescues every candidate class, not just keyword-dense profiles
- Adversarial Trust Layer (Advocate + Skeptic + Verdict) → falsifiable, hallucination-free justifications
- All reasoning is local and template-based → zero external API calls, zero hallucination by design
- Pre-computed indexes + parallel execution → scales to 100K candidates in <5 min on 8-core CPU

JD Understanding & Candidate Evaluation

Target Role: Senior AI Engineer — Founding Team (Redrob AI, Series A)
Key JD Requirements Extracted by JD Parser:

- Experience band: 5–9 years (YOE min 5.0, YOE max 9.0) I Location: Pune / Noida, India (Hybrid)
- REQUIRED: Embeddings-based retrieval (sentence-transformers, FAISS, Pinecone)
- REQUIRED: Vector databases / hybrid search infrastructure
- REQUIRED: Production Python I Evaluation frameworks (NDCG, MRR, MAP, A/B testing)
- NICE-TO-HAVE: LLM fine-tuning (LoRA, QLoRA, PEFT) I Learning-to-rank (XGBoost/neural)
- NICE-TO-HAVE: HR-tech / recruiting-tech background I Distributed systems / large-scale inference

Hard Disqualifiers (JD Stated):

- Consulting-only career (TCS, Wipro, Infosys, Accenture, Cognizant, Capgemini, etc.) → 0.30× hard penalty
- Primary expertise in CV, speech, or robotics without NLP/IR
- No production deployment experience (research-only roles)
- Only recent LangChain demos without pre-LLM ML production experience
- No production code written in last 18 months (architecture-only roles)

Candidate Evaluation Beyond Keyword Matching:

- Ontology graph traversal rescues domain-transfer candidates (RecSys → IR, Search Eng → IR, NLP → Ranking)
- Career trajectory velocity: promotions/year normalized across 100K pool — rewards IC risers, penalizes title inflation
- 23 behavioral platform signals: recency decay, response rate, notice period, open-to-work flag, GitHub activity
- Proficiency trust: Expert=1.0×, Advanced=0.85×, Intermediate=0.65×, Beginner=0.40× I Duration trust: 24+ months = full 1.0×

Ranking Methodology

5-Stage Candidate Ranking Pipeline

Stage 1 — Five Parallel Retrieval Paths:

- Path 1 Semantic: FAISS IVF256 (all-MiniLM-L6-v2, 384-dim), nprobe=32, top-350 by cosine similarity
- Path 2 Keyword: BM25 inverted index + ontology query expansion (skill_map.json), top-350
- Path 3 Ontology: BFS graph traversal from required skills → domain-transfer rescue, top-280, RRF bonus 1.3×
- Path 4 Trajectory: Career pattern vectors (IC-riser, product-co, YOE band), top-220
- Path 5 Signal: 23 behavioral engagement signals (open_to_work, last_active, response_rate, notice_period), top-200, RRF bonus 1.1×

Stage 2 — RRF Fusion + Cross-Encoder Rerank:

- Reciprocal Rank Fusion: rrf_score = Σ (path_bonus / (70 + rank_in_path)) → top-800 deduplicated
- Cross-Encoder: ms-marco-MiniLM-L-6-v2, pairwise JD × candidate, ~80ms/candidate, top-300

Stage 3 — Composite Score Formula:

- final_score = 0.40 × skill_match + 0.30 × career_quality + 0.20 × behavioral + 0.10 × trajectory
- × uncertainty_penalty (0.70–1.00 for sparse profiles) then blended: (0.85 × weighted_sum) + (0.15 × CE_score)
- Min-max normalization into [0.10, 1.00] for discriminative final output

Stage 4 — LLM Blend for top-50 (Qwen2.5-1.5B GGUF):

- Final blend: 0.60 × weighted_sum + 0.25 × CE_score + 0.15 × LLM_score
- Weight invariant validated at import time: WEIGHT_SKILL + WEIGHT_CAREER + WEIGHT_BEHAVIORAL + WEIGHT_TRAJECTORY == 1.0

Configuration Reference

Configuration Reference — config.py (All Centralized, Never Hardcoded)

Retrieval Pool Sizes:

- SEMANTIC_PATH_TOP_K = 350 (Path 1: FAISS) I KEYWORD_PATH_TOP_K = 350 (Path 2: BM25)
- ONTOLOGY_PATH_TOP_K = 280 (Path 3: graph) I TRAJECTORY_PATH_TOP_K = 220 (Path 4: career) I SIGNAL_PATH_TOP_K = 200 (Path 5: signals)
- RRF_POOL_SIZE = 800 (after RRF fusion) I CROSS_ENCODER_TOP_K = 300 I LLM_RERANKER_TOP_N = 50

Scoring Weights (sum = 1.0, validated at import):

- WEIGHT_SKILL = 0.40 I WEIGHT_CAREER = 0.30 I WEIGHT_BEHAVIORAL = 0.20 I WEIGHT_TRAJECTORY = 0.10
- CE_BLEND_FACTOR = 0.15 (cross-encoder blend) I LLM_BLEND_FACTOR = 0.15 (LLM score blend)

RRF Parameters:

- RRF_K = 70 (smoothing constant) I RRF_ONTOLOGY_PATH_BONUS = 1.3 I RRF_SIGNAL_PATH_BONUS = 1.1

Career Quality Modifiers:

- CONSULTING_ONLY_PENALTY = 0.30 I PRODUCT_CO_BONUS = 1.20 I UNCERTAINTY_PENALTY_FLOOR = 0.70

Honeypot Thresholds:

- HONEYPOT_COMPLETENESS_THRESHOLD = 35 I HONEYPOT_SKILLS_STUFFING_COUNT = 15 I HONEYPOT_YOE_DISCREPANCY_YEARS = 4

LLM Model:

- LLM_MODEL_PATH = models/qwen2.5-1.5b-instruct-q4_k_m.gguf I LLM_N_CTX = 512 I LLM_N_THREADS = 8 I LLM_RERANKER_ENABLED = True

Ontology & Skill Graph

Ontology & Skill Graph — Tier-5 Candidate Rescue

skill_map.json: ~38 KB, 1,212 lines I 4 Relationship Types:

7.1 Synonyms (Bidirectional near-equivalents):

- pytorch ↔ [tensorflow, jax, torch, mxnet, ml framework]
- sentence transformers ↔ [sbert, bi-encoder, dense encoder]

7.2 Co-Skills (Directional associations):

- faiss → [vector search, ann search, hnsw, ivf]
- embedding retrieval → [sentence transformers, bi-encoder, dense retrieval]

7.3 Domain Transfers — Key Tier-5 Rescue Paths:

- recommendation systems → [information retrieval, ranking, search]
- recsys → [ranking, ir, vector search]
- nlp → [information retrieval, text search, semantic search]
- search engineering → [information retrieval, faiss, elasticsearch]
- applied machine learning → [information retrieval, ranking]

7.4 Seniority Adjacency:

- Role titles that signal equivalent expertise level for career-history expansion

BFS Graph Traversal: starts from required skills, walks adjacency edges to find domain-transfer candidates not found by keyword or semantic search alone

Scoring Model — Deep Dive

Scoring Model Deep Dive — Four Dimensions

6.1 Skill Match Score (Weight: 40%):

- Required skill weight 2.0× vs nice-to-have 1.0× I Proficiency: Expert=1.00, Advanced=0.85, Intermediate=0.65, Beginner=0.40
- Duration trust: zero-duration floor=0.50, full trust at 24+ months=1.0× I Endorsement boost: log-scaled, max +0.10 (capped at 50 endorsements)
- Ontology partial credit: adjacent skill earns 0.60× of full score (domain-transfer rescues)

6.2 Career Quality Score (Weight: 30%):

- Product-company (SaaS, FinTech, AI/ML, Food-tech): 1.20× multiplier I Consulting-only: 0.30× hard penalty
- YOE ideal band (5–9 years): 1.0× I YOE outside [4, 12]: tapers to 0 I Trajectory velocity: promotions/year, percentile-normalized
- Location bonus: Noida/Pune +0.08 I Delhi/Gurgaon +0.05 I Hyderabad/Mumbai +0.04 I Bangalore +0.03

6.3 Behavioral Score (Weight: 20%) — 23 Platform Signals:

- last_active_date (exponential decay λ=0.005): sub-weight 0.25 I recruiter_response_rate: 0.20 I open_to_work_flag: 0.15
- notice_period_days (ideal ≤30): 0.15 I github_activity_score: 0.10 I profile_completeness_score: 0.10 I interview_completion_rate: 0.05
- Special: github_activity_score=-1.0 → no GitHub linked (neutral) I offer_acceptance_rate=-1.0 → excluded from scoring

6.4 Trajectory Score (Weight: 10%):

- Promotions per year, normalized across entire 100K candidate pool
- Rewards IC risers (individual contributor growth) I Penalizes frequent company-switching without growth
- Distinguishes genuine career velocity from title inflation

LLM Reranker

LLM Reranker — scoring/llm_reranker.py

Model Specifications:

- Model: Qwen2.5-1.5B-Instruct | Format: 4-bit quantized GGUF (Q4_K_M) | Backend: llama-cpp-python
- Size: ~1.1 GB on disk | RAM usage: ~1.0 GB | GPU: NOT required (CPU-only) | Network: NOT required at ranking time

Coverage & Runtime:

- Applied to top-50 candidates only — generates rich, human-quality justifications
- Runtime: ~200–400ms per candidate on 8 CPU cores | Top-50 total: ~200 seconds
- Candidates ranked 51–100: rule-based reasoning (still fact-grounded from actual profile fields)

Background Pre-Warm Optimization:

- LLM worker pool starts loading in background thread DURING cross-encoder stage (~10s)
- By the time 10-second cross-encoder finishes, Qwen2.5 is already hot — eliminating model load latency from critical path
- Effective saving: ~15–30 seconds eliminated from total ranking time

Final LLM Composite Blend Formula:

- final_score = 0.60 × weighted_sum (skill+career+behavioral+trajectory)
- + 0.25 × cross_encoder_score (pairwise JD×candidate relevance)
- + 0.15 × llm_score (Qwen2.5 relevance judgment)

Hallucination Guard: LLM prompt is constructed from structured profile fields only — not free-text generation. All output is fact-anchored.

Explainability & Data Validation

Adversarial Trust Layer: Advocate → Skeptic → Verdict

How Ranking Decisions Are Explained (No LLM in Trust Layer):

- Advocate Scan: checks required skill coverage, product-company trajectory, assessment scores >70, last_active <30d, strong response rate → tags HIGH/MEDIUM/LOW confidence
- Skeptic Scan: checks missing required skills, consulting-only career, last_active >90d, notice_period >60d, low response rate, incomplete profile → tags HIGH/MODERATE/LOW risk
- Verdict: ROBUST (0 HIGH risks), CONTESTED (1 HIGH risk), FRAGILE (≥2 HIGH risks)
- Confidence % = clamp((positive_mass / (positive + negative)) × 100, 10, 95)
- Falsifiability contract: 2–3 specific, verifiable conditions that would change the ranking

Hallucination Prevention — Structurally Impossible:

- All reasoning sentences reference actual profile fields — no generative fabrication
- Template-based reasoning_generator.py: fact-grounded output only, no free-text LLM in trust layer
- For top-50: Qwen2.5-1.5B local GGUF generates rich justifications from structured profile data — not from imagination
- Example output: "7 years applied ML at product companies (Swiggy, Razorpay); strong FAISS + embedding retrieval match; notice period 45 days is above ideal. Verdict: CONTESTED — would flip to FRAGILE if response rate confirmed <15%."

Suspicious / Low-Quality Profile Handling (Honeypot Filter):

- Rule 1 Salary Anomaly: expected_salary_min > expected_salary_max → flagged O(1)
- Rule 2 Profile Stuffing: completeness_score <35 AND skill_count >15 → flagged O(1)
- Rule 3 YOE Discrepancy: sum(career_months)/12 − years_of_experience >4 years → flagged O(jobs)
- Rule 4 Expert Skill Zero Duration: proficiency=expert AND duration_months <1 → flagged O(skills)
- Rule 5 Skill-Career Mismatch: <60% of claimed skills appear in career descriptions → flagged O(skills×text)
- Target: honeypot rate <5% in top-100 (well below 10% disqualification threshold)

End-to-End Workflow

Complete Workflow: JD Input → Ranked Output

PHASE 1 — PRE-COMPUTATION (Online, Run Once):

- Install: pip install -r requirements.txt I python -m spacy download en_core_web_sm
- Download LLM: python precompute_llm.py
- Build indexes: python build_indexes.py or download from Google-Drive already built by us.
- Outputs: faiss.index (~150 MB IVF256), bm25.pkl (~30 MB), features.npy (23 signals), trajectory.npy, honeypots.pkl

PHASE 2 — RANKING (≤5 Minutes, No Network):

- [1] Honeypot Filter: O(1) set lookup removes fake profiles before any expensive computation
- [2] Five Retrieval Paths: concurrent via ThreadPoolExecutor(5) → 3–5× speedup over serial I total ~1.5s
- [3] RRF Fusion: up to 1,400 raw results → 800 deduplicated, ranked by rrf_score
- [4] Cross-Encoder Rerank: ms-marco-MiniLM-L-6-v2, top-300 → top-300 reranked I ~10s
- [5] Composite Scorer: 4-dimension weighted sum + CE blend → min-max norm to [0.10, 1.00] I ~0.5s
- [6] Trust Layer (Advocate + Skeptic + Verdict): template-based, all top-100 I ~1s
- [7] LLM Justification: Qwen2.5-1.5B GGUF, top-50 candidates only I ~200s
- [8] Assemble + Validate + Write: submission.csv, 100 rows, ranks 1–100 I ~0.1s

Run Command: python rank.py –input data/candidates.jsonl –output output/submission.csv –top-k 100
Output:output/submission.csv

System Architecture

PHASE 1 — PRE-COMPUTATION

Offline · Run Once

candidates.jsonl	100K profiles (~487 MB)
Candidate Parser	→ CandidateFeatureVector
JD Parser	→ JDIntent (structured)
FAISS Index Builder	→ faiss.index (~150 MB, IVF256)
BM25 Index Builder	→ bm25.pkl (~30 MB)
Feature Store	→ features.npy (23 signals)
Trajectory Builder	→ trajectory.npy
Honeypot Registry	→ honeypots.pkl

→

PHASE 2 — RANKING

≤ 5 Minutes · No Network · CPU Only

1	**Honeypot Filter**	O(1) set lookup — remove fakes
2	**5 Retrieval Paths**	Parallel threads — top-350/280/220/200
3	**RRF Fusion**	Reciprocal Rank Fusion → top-800
4	**Cross-Encoder Rerank**	ms-marco-MiniLM-L-6-v2 — top-300
5	**Composite Scorer**	Skill×0.4 + Career×0.3 + Behavioral×0.2 + Traj×0.1
6	**Normalization**	Min-max → [0.10, 1.00]
7	**Trust Layer**	Advocate → Skeptic → Verdict
8	**LLM Justification**	Qwen2.5-1.5B GGUF, top-50 only
9	**Output**	submission.csv — top-100 ranked candidates

Project Structure

Project Structure — RAGnarok/
Root Level:

- config.py: central config (all constants, weights, paths) I rank.py: CLI entry point → submission.csv
- precompute.py: offline index builder (Phase 1) I precompute_llm.py: downloads LLM model (one-time)

pipeline/ — Orchestration:

- candidate_parser.py: raw JSON → CandidateFeatureVector I jd_parser.py: JD markdown → JDIntent
- runner.py: orchestrates all 10 pipeline stages I schemas.py: Pydantic-like dataclasses + weight invariants

indexing/ — Index Building:

- faiss_builder.py (IVF256) I bm25_builder.py I feature_store.py (23 signals) I trajectory_builder.py I honeypot_registry.py (5 rules)

ontology/ — Skill Graph:

- skill_map.json (~38 KB, 1,212 lines) I query_expander.py I graph_traversal.py (BFS)

retrieval/ — Five Paths + Fusion:

- semantic_path.py I keyword_path.py I ontology_path.py I trajectory_path.py I signal_path.py I rrf_fusion.py

scoring/ — Multi-Dimensional Scoring:

- skill_match.py I career_quality.py I behavioral.py I trajectory.py I composite.py I cross_encoder.py I honeypot_filter.py I llm_reranker.py

trust/ — Adversarial Layer:

- advocate.py I skeptic.py I verdict.py I reasoning_generator.py

Results & Performance

Runtime Budget — All Targets Met on 8-Core CPU, 16 GB RAM

Phase 2 Runtime Breakdown (Indexes Pre-Built):

- Load FAISS index: ~2s (IVF256, ~150 MB)
- Load BM25 + feature store: ~6s (pickle + numpy)
- Encode JD query: ~0.1s (1 text → 384-dim vector)
- Five retrieval paths (parallel): ~1.5s (threads overlap I/O)
- RRF fusion: ~0.05s (pure Python) I Honeypot filter: ~0.01s (O(1) set lookup)
- Cross-encoder (300 candidates): ~10s (~80ms × 300) I LLM pre-warm: 0s additional (runs during CE stage)
- Composite scoring: ~0.5s (vectorized numpy) I Trust layer (top-100): ~1s (rule-based)
- LLM Reasoning (top-50): ~200s (Qwen2.5-1.5B, CPU-only) I CSV write + validate: ~0.1s

TOTAL: ~230 seconds — Well within 5-minute budget

RAM Budget (Total Peak: ~2.9 GB, well within 16 GB constraint):

- Qwen2.5-1.5B Q4 GGUF: ~1.0 GB I FAISS + BM25 + indexes: ~250 MB
- Candidate objects (100K): ~800 MB I PyTorch + sentence-transformers: ~500 MB I OS + Python baseline: ~300 MB

Ranking Quality Design:

- Output: exactly 100 candidates, ranks 1–100 each unique, scores monotonically non-increasing
- Honeypot rate in top-100 <5% (spec threshold: 10%) I All reasoning non-empty, all IDs match CAND_XXXXXXX format
- Required skill weight: 2.0× vs nice-to-have; product-company bonus 1.20×; consulting-only penalty 0.30×

Output Format

Output Format — submission.csv
Output File: output/submission.csv

Columns:

- candidate_id (string): Format CAND_XXXXXXX (7 digits) — unique identifier for each candidate
- rank (int): 1–100, unique per row, monotonically increasing
- score (float): 0.0–1.0, non-increasing with rank (min-max normalized to [0.10, 1.00])
- reasoning (string): 1–2 sentence fact-grounded explanation referencing actual profile fields

Validation Rules (checked before writing):

- Exactly 100 data rows
- Ranks 1–100 each appear exactly once
- Scores are monotonically non-increasing
- All candidate_ids match CAND_XXXXXXX format
- No duplicate candidate_ids
- Reasoning is non-empty for all rows
- Honeypot rate in top-100 <10%

Example Reasoning Output: “7 years applied ML at product companies (Swiggy, Razorpay); strong FAISS + embedding retrieval match; notice period 45 days above ideal. Verdict: CONTESTED — would flip to FRAGILE if response rate confirmed <15%.”

Technologies Used

Technology Stack — Fully Pinned, CPU-Only, No External APIs

Core ML / AI Libraries:

- sentence-transformers 3.4.1: bi-encoder (all-MiniLM-L6-v2) for FAISS embeddings + cross-encoder (ms-marco-MiniLM-L-6-v2) for reranking
- torch 2.5.1: CPU-only PyTorch backend for transformer inference
- faiss-cpu 1.8.0: dense vector index with IVF256 (inverted file, 256 centroids) for ~150 MB, fast ANN search
- llama-cpp-python 0.3.x: local GGUF inference engine for Qwen2.5-1.5B-Instruct (4-bit quantized, Q4_K_M)

Retrieval / NLP:

- rank-bm25 0.2.2: BM25 sparse keyword retrieval with ontology query expansion
- spacy 3.7.4: JD natural language parsing to extract required skills, disqualifiers, YOE range

Data / Infrastructure:

- numpy 1.26.4: feature arrays, vectorized composite scoring, trajectory vectors
- pandas 2.2.3: CSV output (submission.csv), structured data manipulation
- pydantic 2.x: schema validation for all dataclass objects (SkillRecord, CareerEntry, JDIntent, RankedCandidate, etc.)
- fastapi 0.115.14: optional sandbox REST API for interactive testing

Hardware: Python 3.11.2 I Windows 12 I Lenovo Ideapad i5 I 8 cores I 16 GB RAM I CPU only I No GPU I No internet during ranking

Why selected: all packages are CPU-native, offline-capable, and avoid heavy cloud dependencies; FAISS IVF256 chosen for balance of speed and memory at 100K scale

Submission Assets

Hackathon Submission Assets

Team Information:

- Team Name: RAGnarok I Event: Redrob Intelligent Candidate Discovery & Ranking Hackathon
- Team Leader: Krishna H Zalavadiya (ML Engineer) I Team Member: Daksh R Jain (Data Engineer & Backend Engineer)
- Primary Contact: Daksh R Jain I dakshjain737@gmail.com
- Secondary Contact:Krishna H Zalavadiya I krish77zalavadiya@gmail.com

Code Repository:

- GitHub: https://github.com/krishna-zalavadiya/RAGnarok
- Platform: Windows 12, Lenovo Ideapad i5, 8 cores, 16 GB RAM I Python 3.11.2

Reproduce Command:

- python rank.py –candidates ./candidates.jsonl –out ./submission.csv

AI Tools Disclosure:

- Claude: architecture design & debugging assistance
- ChatGPT: design suggestions

Output Validation:

- python scripts/validate_output.py output/submission.csv
- Checks: 100 data rows, ranks 1–100 unique, monotonic scores, CAND_XXXXXXX format, no duplicates, non-empty reasoning, honeypot rate <10%

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU